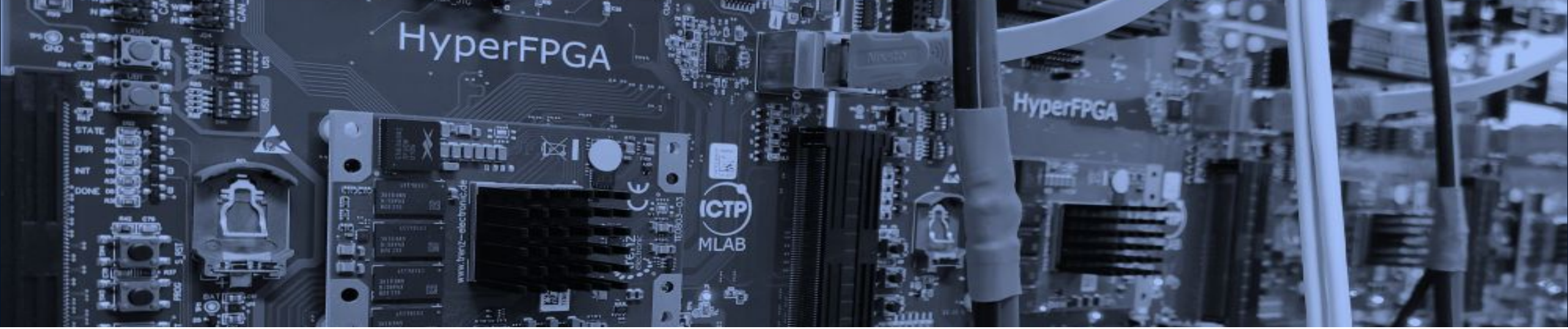


AN EVOLUTIONARY APPROACH TO MODELING A CHAOTIC DOUBLE PENDULUM

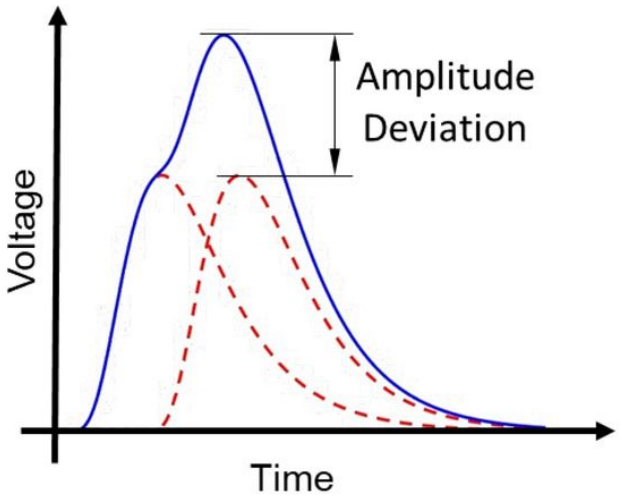
From Numerical Integration to Physics-Informed Neural Networks

Maynor Ballina · Maria Liz Crespo
Multidisciplinary Laboratory (MLAB), STI Unit, ICTP

AGENDA

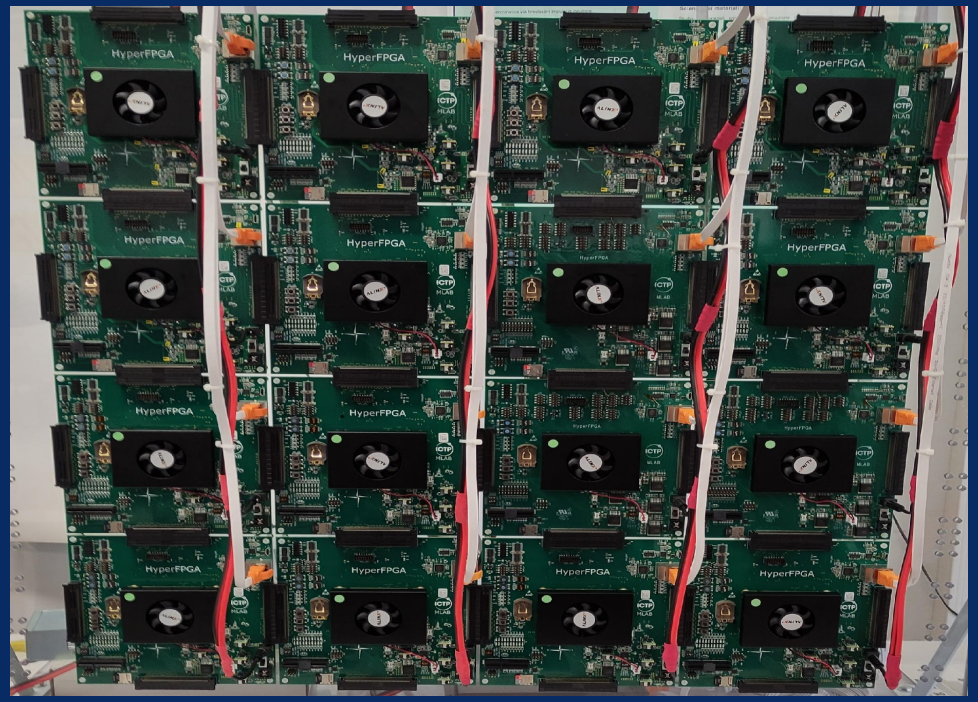
Outline

- The Double Pendulum — a chaotic benchmark
- Classical Numerical Integration — RK4
- Data-Driven Learning — MLP
- Physics-Informed Learning — PINN
- Parametric PINN — PPINN
- Compressing Models — Quantization
- Other Models — cPINN, gPINN, xPINN
- Conclusions



Goal of this Lab

Understand **why** each new method was developed, not just how it works, by following the natural evolution of differential-equation solvers: from numerical integration to data-driven and physics-informed machine learning.



Maynor Ballina
MLAB · ICTP

SECTION 1

The Double Pendulum

Maynor Ballina
MLAB · ICTP

What Is a Double Pendulum?

- Two rigid, massless rods of length L_1, L_2, each ending in a point mass m_1, m_2; the second rod hangs from the first.
- State vector: $\mathbf{q} = [\theta_1, \omega_1, \theta_2, \omega_2]$, two angles and two angular velocities.
- Governed by **coupled, nonlinear ODEs** derived from Lagrangian mechanics (kinetic, potential energy).
- Only 2 degrees of freedom, yet no closed-form analytical solution exists.

NEWTON'S NOTATION

$$\omega_1 = d\theta_1/dt \quad \omega_2 = d\theta_2/dt$$

Why This System?

It is the simplest mechanical system that reliably exhibits **deterministic chaos**: fully deterministic equations, yet practically unpredictable over long horizons. That combination makes it an ideal stress test for both numerical solvers and machine-learning surrogates.

Used throughout this lab with $m_1=m_2=1$ kg, $L_1=L_2=1$ m, $g=9.81$ m/s².

Maynor Ballina
MLAB · ICTP

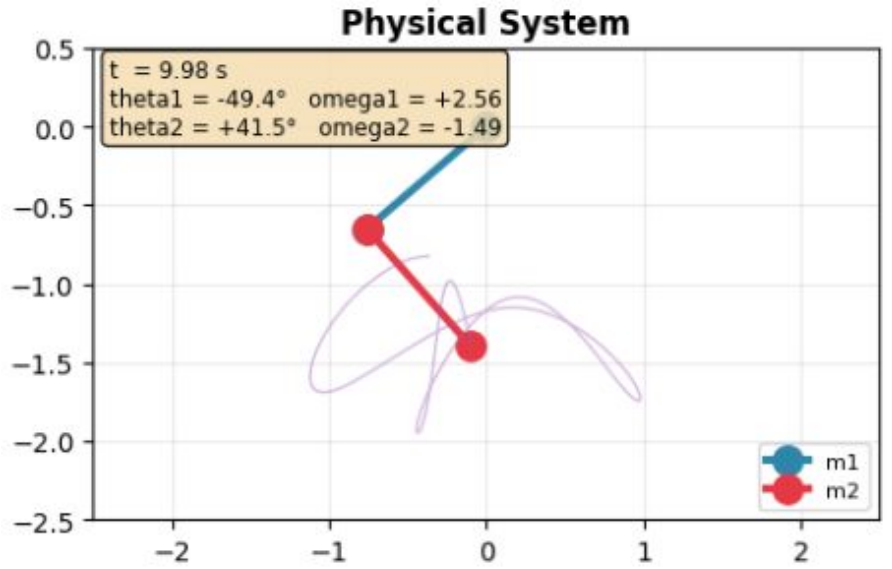
Equations of Motion

ANGULAR ACCELERATIONS ($\Delta = \Theta_1 - \Theta_2$)

$$d\omega_1/dt = [\, -g(2m_1+m_2)\sin\theta_1 - m_2 g \cdot \sin(\theta_1-2\theta_2) - 2\sin\Delta \cdot m_2(\omega_2^2 L_2 + \omega_1^2 L_1 \cos\Delta)\,] \,/\, [\, L_1(2m_1+m_2-m_2\cos 2\Delta)\,]$$

$$d\omega_2/dt = [\, 2\sin\Delta \cdot (\omega_1^2 L_1(m_1+m_2) + g(m_1+m_2)\cos\theta_1 + \omega_2^2 L_2 m_2 \cos\Delta)\,] \,/\, [\, L_2(2m_1+m_2-m_2\cos 2\Delta)\,]$$

These equations are **stiff and strongly nonlinear** in θ_1, θ_2: the coupling terms $\sin\Delta$ and $\cos\Delta$ mean a tiny change in either angle redistributes energy between the two rods in a highly nonlinear way, the mathematical root of the system's chaotic behavior.



Maynor Ballina
MLAB · ICTP

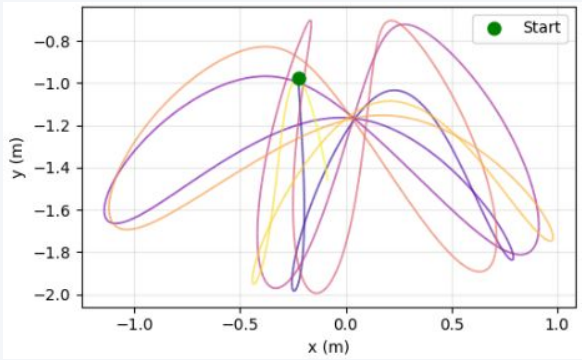
Why Is It Chaotic?

- **Deterministic chaos**: identical equations, but output depends extremely sensitively on the initial condition.
- Two trajectories starting only 0.01 rad (~0.6°) apart diverge **exponentially**, not linearly.
- Quantified by the **maximal Lyapunov exponent** $\lambda > 0$:

  separation grows as $|\delta(t)| \approx |\delta(0)| \cdot e^{(\lambda t)}$.
- Practical consequence: beyond a finite *prediction horizon* (~1/λ), forecasting becomes meaningless, for any method, not only neural networks.

The Butterfly Effect, Concretely

Three RK4 runs with initial θ_1 differing by only 0.01 rad track almost identically for the first 2–3 seconds, then visibly split apart, and by t = 10 s show completely uncorrelated swinging patterns, despite being solved by the *same* deterministic integrator.



This single property will resurface at every later stage of the lab: it is the reason MLP, PINN and PPINN *all* eventually lose accuracy, regardless of architecture.

SECTION 2

Classical Numerical Integration — RK4

The RK4 Method

- Given θ̇ form ODE dq/dt = f(t, q), RK4 advances one step h using **4 slope estimates** combined with weighted averaging.
- 4th-order accurate: local truncation error is $O(h^5)$, global error $O(h^4)$.
- Deterministic and physically exact up to floating-point precision, used here as the **ground-truth reference** for every learning method.

UPDATE RULE

$$q(t+h) = q(t) + (h/6)(k_1 + 2k_2 + 2k_3 + k_4)$$

The Four Slopes

$k_1 = f(t, q)$

$k_2 = f(t+h/2, q+h/2 \cdot k_1)$

$k_3 = f(t+h/2, q+h/2 \cdot k_2)$

$k_4 = f(t+h, q+h \cdot k_3)$

k_1, k_4 sample the slope at the interval endpoints; k_2, k_3 sample it (twice) at the midpoint — this is what pushes local error from $O(h^2)$ (Euler) down to $O(h^5)$.

Maynor Ballina
MLAB · ICTP

Simulation Parameters — What Each One Varies

Parameter	Meaning	What Varying It Changes
θ_1^0, θ_2^0 (deg)	Initial rod angles	Selects which trajectory in state space is followed — the dominant lever on chaotic behavior
ω_{10}, ω_{20} (rad/s)	Initial angular velocities	Adds initial kinetic energy / spin to each rod
L_1, L_2 (m)	Rod lengths	Rescales the natural oscillation period and coupling strength between rods
m_1, m_2 (kg)	Point masses	Changes inertia and the mass ratio that governs energy transfer at the joint
g (m/s²)	Gravitational acceleration	Scales the restoring force / overall oscillation frequency
T (s)	Total simulation time	Longer T pushes further past the chaotic prediction horizon
dt (s)	Integration step	Smaller dt → higher accuracy but proportionally more compute (O(1/dt) steps)

Building the Reference Dataset

generate_dataset() draws **50 random initial conditions** and integrates each with RK4.

- T = 10 s per trajectory, dt = 0.01 s → 1000 time steps, seed = 42 (reproducible).
- Output tensor: (50 trajectories × 1000 steps × 4 state variables).

This dataset becomes the **supervised-learning ground truth** for every neural model in the rest of the lab.

The Core Limitation

RK4 is accurate and physically exact, but it is a **per-trajectory solver**: every new initial condition, or every new evaluation point, requires rerunning the full sequential integration. There is no way to “query” the solution instantly.

This single limitation motivates **every** method that follows in this lab.

Maynor Ballina
MLAB · ICTP

RK4 — Strengths & Limitations

Strengths	Limitations
4th-order accurate, $O(h^5)$ local error	Sequential, cannot parallelize across time
Physically exact (no learned approximation)	Must rerun the full solve for every new IC or parameter set
No training required, no data needed	Not differentiable end-to-end / not embeddable in an ML pipeline
Deterministic, reproducible reference	Cost grows linearly with 1/dt and with number of queried ICs

Discussion Questions

Q: Why is RK4 preferred over the simpler Euler method?
Euler uses a single first-order slope estimate (local error $O(h^2)$); RK4 combines four slope evaluations per step for local error $O(h^5)$, so it reaches the same accuracy with far larger steps and far less accumulated drift.

Q: What happens if you double the step size h?
Local truncation error scales as $O(h^5)$, so doubling h increases per-step error roughly 32×; for a chaotic system this error is then exponentially amplified, causing visibly wrong trajectories much sooner.

Q: Why does every new initial condition require rerunning the solver?
RK4 propagates the state sequentially, step by step, from a specific q(0); it has no notion of a general closed-form solution, so a different starting point simply requires a fresh integration from t=0.

SECTION 3

Data-Driven Learning — MLP

Maynor Ballina
MLAB · ICTP

Motivation: A Fast Surrogate

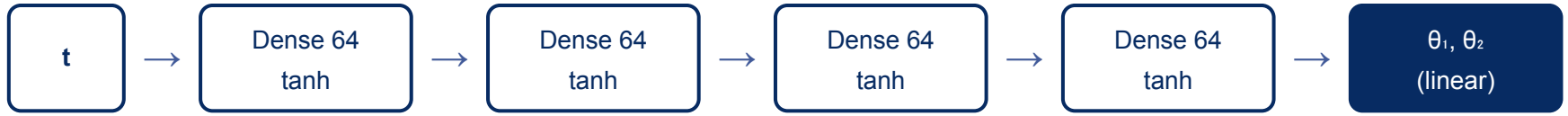
- RK4 must rerun sequentially for every new initial condition, too slow for real-time or repeated queries.
- Idea: train a neural network $f(t; \theta) \approx (\theta_1(t), \theta_2(t))$ once, then get predictions **instantly** at any t.
- A Multi-Layer Perceptron (MLP) is the simplest such surrogate: a stack of fully-connected layers with nonlinear activations, trained purely from RK4-generated data.
- This is **pure supervised learning**, the network sees only (t, θ_1, θ_2) pairs, never the governing equations.

Universal Approximation

A feed-forward network with enough hidden units can approximate any continuous function on a compact domain (Cybenko, 1989; Hornik, 1991). In principle, an MLP *can* represent $\theta_1(t)$, $\theta_2(t)$, the open question is whether it *generalizes* beyond the exact points it was trained on.

MLP Architecture

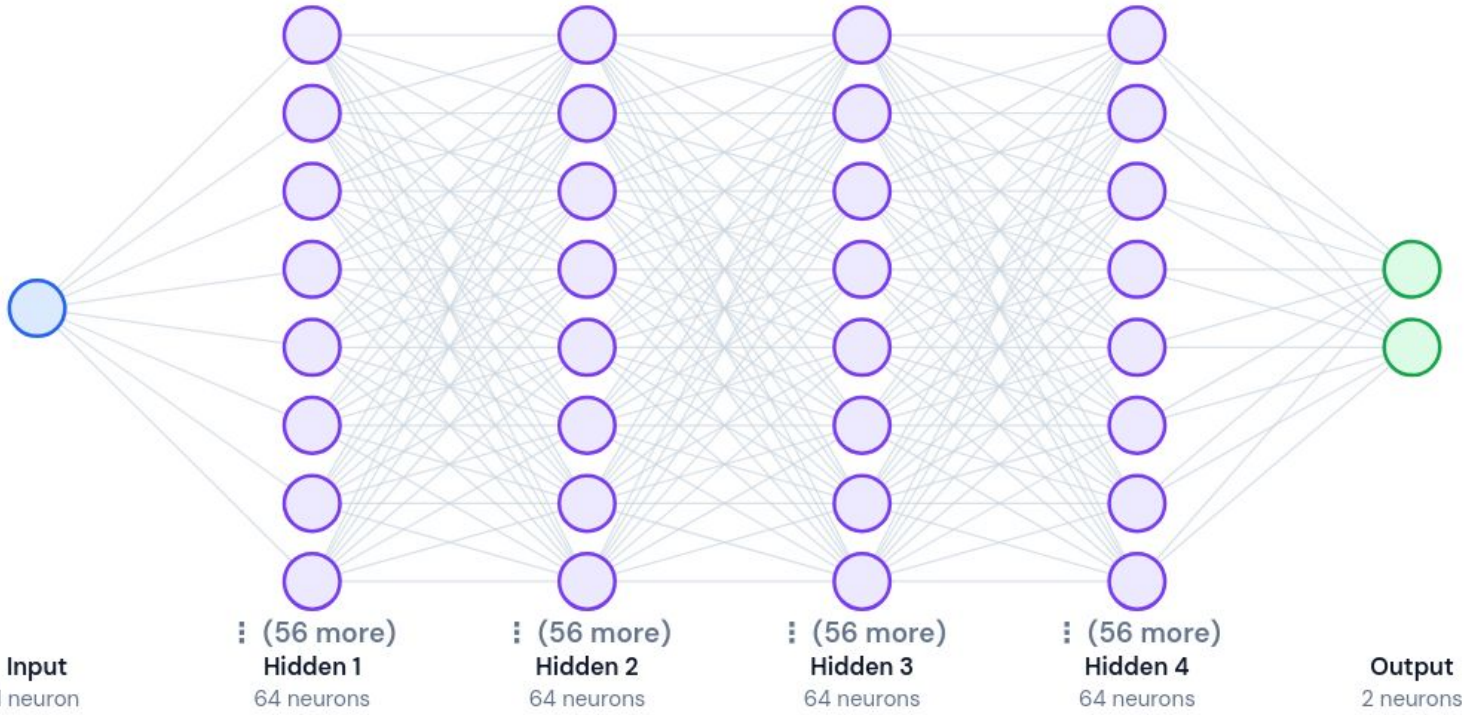
Input → Output Mapping



Layer-by-Layer

1 input (t) → 4 hidden Dense layers × 64 units, tanh → 1 linear output layer with 2 units (θ_1, θ_2).

tanh is smooth and infinitely differentiable needed later when PINN differentiates the network twice.



Why This Architecture?

The lab keeps **the same architecture** (4×64, tanh) for MLP, PINN and the PINN half of PPINN. Only the training objective and input size change, so accuracy differences reflect the method, not extra model capacity.

Maynor Ballina
MLAB · ICTP

Training Process & Hyperparameters

Hyperparameter	Value
Epochs (max)	5,000
Optimizer	Adam, lr = 1e-3
Batch size	64
Early-stop patience	200 epochs
Hidden layers × units	4 × 64
Training trajectory	1 of 50

Loss & Optimization Elements

OBJECTIVE

$$L\_MLP = mean[(\hat{\theta}_1 - \theta_1)^2 + (\hat{\theta}_2 - \theta_2)^2]$$

Pure MSE regression, minimized by **Adam** (adaptive per-parameter learning rate).

Early Stopping on validation loss (patience 200), halts training and restores best weights once it stops improving.

Reduce LR On Plateau (factor 0.5), halves the learning rate when progress stalls, down to a 1e-6 floor.

Maynor Ballina
MLAB · ICTP

How Do We Know If a Model Is Good?

Metric	Definition	What It Tells Us
RMSE	√(mean squared error)	Penalizes large deviations heavily — sensitive to occasional big misses
MAE	mean(|prediction − truth|)	Average error magnitude, robust to outliers
Error vs. time	|error(t)|, plotted over t	Reveals *where* and *how fast* a model diverges from RK4
Inference time	ms per sample / per batch	Deployment cost — the whole reason to use a surrogate
Physics residual	ODE left-hand − right-hand side, evaluated on predictions	Physical consistency: does the output actually obey Newton's laws?
Generalization	Error on unseen ICs / t ranges	Does the model only memorize, or truly learn the dynamics?

MLP Results

Training Behavior

Train and validation MSE (log scale) both fall steadily over 5,000 epochs and track each other closely, the model fits the single training trajectory well, with early stopping preventing overt overfitting on this short 10 s window.

Fit vs. RK4

Overlaying MLP predictions on the RK4 reference for $\theta_1(t)$, $\theta_2(t)$ shows a close match *inside* the training window [0, 10] s, as expected for interpolation.

Metric	MLP
RMSE θ_1 / θ_2 (rad)	0.813 / 0.968
MAE θ_1 / θ_2 (rad)	0.732 / 0.867
Train time (s)	9.8
Inference, single (ms)	1.48
Inference, batch (ms)	2.81

Maynor Ballina
MLAB · ICTP

Discussion Questions

Q: The MLP fits the training data well. Does that mean it understands the physics?
No. It minimizes MSE against sampled points only; nothing in its loss enforces Newton's equations, so it can fit the data while producing physically inconsistent derivatives or violating energy conservation.

Q: What happens if you query the MLP at t > 10 s (outside training)?
Predictions degrade rapidly — the network has no mechanism to extrapolate a periodic-looking but chaotic signal; outside its training window it typically flattens, drifts, or oscillates unrealistically.

Q: How much data is needed to train an acceptable model?
Enough densely-sampled points to cover the oscillatory structure of the target trajectory; too few samples under-resolve fast swings, while chaotic systems also mean more data cannot fix long-horizon divergence.

MLP — Strengths & Limitations

Strengths	Limitations
Extremely fast inference (~1.5 ms/sample) once trained	Requires a large labeled dataset (RK4 ground truth)
Simple to implement and train	No knowledge of the governing physics
Differentiable, GPU/FPGA-friendly	Poor extrapolation outside the training window
Easy to batch for many simultaneous queries	May violate energy conservation / physical laws

Universal approximation theorem: Cybenko (1989), Mathematics of Control, Signals and Systems; Hornik (1991), Neural Networks.

SECTION 4

Physics-Informed Neural Networks

What Is a PINN?

- A **Physics-Informed Neural Network** is a neural network trained so its output satisfies a known governing differential equation, not just labeled data.
- Introduced by Raissi, Perdikaris & Karniadakis (2019) as a way to embed physics directly into the loss function.
- Key mechanism: **automatic differentiation** (autodiff) computes exact derivatives of the network output with respect to its input(s), no finite-difference grid needed.
- The ODE/PDE residual, evaluated at sampled *collocation points*, becomes an extra term the optimizer must drive toward zero.

Core Idea in One Line

Same network, same architecture as an MLP, but the **loss function** now also penalizes disagreement with the governing equations, turning the network into an approximate PDE/ODE solver.

What Are PINNs Used For?

Fluid dynamics: solving Navier–Stokes flows from sparse sensor data.

Heat transfer & materials science: inverse problems, estimating unknown material parameters.

Biomedical engineering: cardiovascular flow modeling, tumor growth.

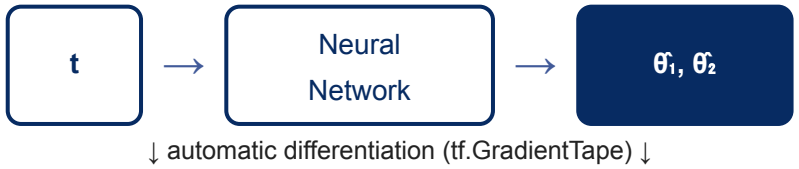
Geophysics: seismic wave inversion with limited surface measurements.

Quantum mechanics & structural engineering: solving high-dimensional PDEs where mesh-based solvers are expensive.

Common Thread

PINNs shine when (a) the governing equation is known, and (b) labeled data is **scarce or expensive** to obtain, the physics term substitutes for the data a purely data-driven model would otherwise need.

How a PINN Works



Physics Residual

Differentiate $\hat{\theta}$ twice w.r.t. t to get predicted acceleration; compare against the ODE right-hand side evaluated at the same point.

RESIDUAL

$$R(t) = d^2\hat{\theta}/dt^2 - f\_ODE(\hat{\theta}, d\hat{\theta}/dt)$$

Composite Loss

TOTAL LOSS

$$L = \lambda\_d \cdot L\_data + \lambda\_p \cdot L\_physics + \lambda\_ic \cdot L\_IC$$

L_physics = mean(R(t)²) over collocation points, points where *no labeled data* is required, only the equation itself.

Maynor Ballina
MLAB · ICTP

Advantages & Disadvantages

Advantages	Disadvantages
Embeds known physics → physically consistent outputs	Non-convex, multi-term loss → harder to optimize than plain MSE
Needs far less labeled data than a pure MLP	Balancing λ_data, λ_phys, λ_ic weights is delicate and problem-specific
Mesh-free: works with scattered collocation points	Training is much slower than supervised regression
Can solve inverse problems (infer unknown parameters)	Struggles with high-frequency / chaotic / multi-scale dynamics
Differentiable end-to-end (autodiff, no numerical grid error)	Still requires knowing the governing equation explicitly

PINN Training Hyperparameters

Hyperparameter	Value
Epochs	10,000
Optimizer	Adam, lr = 1e-3
Hidden layers × units	4 × 64 (identical to MLP)
Collocation points	1,000, uniform over [0, 10] s
λ_data / λ_phys / λ_ic	1.0 / 1.0 / 10.0

λ_ic is weighted 10× higher than the other terms, anchoring t=0 tightly is critical, since every later prediction implicitly builds on it.

PINN Results

Training Behavior

All four loss terms (total, data, physics, IC) fall together over 10,000 epochs: physics loss converges more slowly than data loss, reflecting the harder optimization landscape of the second-derivative residual.

Trajectory & Error-over-Time

On its trained IC, the PINN tracks RK4 closely at first; the error-vs-time curve rises gradually rather than fitting a fixed pointwise pattern, showing physically smoother divergence than the plain MLP.

Metric	PINN
RMSE θ_1 / θ_2 (rad)	0.891 / 1.048
MAE θ_1 / θ_2 (rad)	0.772 / 0.913
Train time (s)	87.3
Inference, single (ms)	1.75

Maynor Ballina
MLAB · ICTP

Validation on a New Initial Condition

- The notebook lets you re-run the PINN on a **different** IC (e.g. $\theta_1^0=90°$, $\theta_2^0=90°$) without retraining.
- Because the PINN was optimized for one fixed IC only, performance **visibly degrades** on the new one.
- This exposes the PINN's core limitation in practice, motivating the Parametric PINN in the next section.

Expected Outcome

Predicted $\theta_1(t)$, $\theta_2(t)$ diverge from the new RK4 reference almost immediately, the network learned *one specific solution curve* of the ODE, not the general solution operator across initial conditions.

PINN · IN THIS LAB

Discussion Questions

Q: Why is the IC loss needed even though we have data at t=0?
The data loss already includes t=0, but it is just one point among thousands, easily outweighed; a dedicated, heavily-weighted IC term ensures the anchor point is matched precisely, since every later prediction depends on it.

Q: The PINN uses collocation points — what are they and why?
Collocation points are locations in the input domain (here, times in [0,10] s) where the ODE residual is evaluated and penalized, without needing any labeled RK4 data there — they let the network learn where data does not cover.

Q: The PINN is still trained for one fixed IC. What limitation does this impose?
It only generalizes along the time axis for that one trajectory; any new set of initial conditions requires retraining from scratch, exactly like RK4 needing a fresh integration.

SECTION 5

Parametric PINN (PPINN)

PPINN

Motivation & Extended Architecture

Every new IC requires retraining the PINN. The **Parametric PINN** fixes this by feeding the initial condition *into the network as an input*, so one trained model covers a whole family of trajectories.

What Changed

Input grows from 1 (t) to 5 (t, θ_1^0, ω_1^0, θ_2^0, ω_2^0); output and hidden architecture stay identical.

Parameterized IC New Loss

$$L = \lambda\_d \cdot L\_data + \lambda\_p \cdot L\_physics + \lambda\_ic \cdot L\_IC + \lambda\_bc \cdot L\_BC$$

PPINN Training

Hyperparameter	Value
Epochs	20,000
Optimizer	Adam, lr = 1e-3
Hidden layers × units	4 × 64
Collocation pts / trajectory	200
Trajectories per mini-batch	8
λ_data / λ_phys / λ_ic	1.0 / 1.0 / 10.0

Trained on All 50 Trajectories at Once

Unlike the single-IC PINN, the PPINN samples mini-batches of 8 different trajectories per step, each contributing its own data, physics and IC loss, the network must fit an entire *family* of solutions simultaneously.

Maynor Ballina
MLAB · ICTP

PPINN Results

Training Behavior

With 4× the epochs of the PINN and a much harder multi-trajectory objective, total loss decreases steadily but training takes considerably longer in wall-clock time.

Metric	PPINN
RMSE θ_1 / θ_2 (rad)	0.804 / 0.931
MAE θ_1 / θ_2 (rad)	0.710 / 0.794
Train time (s)	392.7
Inference, single (ms)	1.55

Real values from *metrics_report.csv*. Despite learning a whole family of trajectories, PPINN achieves the **lowest RMSE/MAE of all four core methods** — the multi-trajectory training acts as a form of regularization.

Maynor Ballina
MLAB · ICTP

Discussion Questions

Q: Why does adding the IC as an input allow generalization across many trajectories?
The network now learns a solution operator $f(t, q_0) \rightarrow$ state, mapping any initial condition (within the training distribution) to its trajectory, instead of memorizing one fixed curve — it interpolates across the IC space the same way it interpolates across time.

Q: Would this approach work if the IC range was very wide? What would happen?
A very wide IC range would require far more collocation points and network capacity to cover the enlarged input space well; coverage would thin out, and accuracy on sparsely-sampled regions of that space would degrade — the same interpolation-vs-extrapolation trade-off as time, now in IC-space too.

Q: How does the total training cost compare to training one PINN per IC?
One PPINN (392.7 s here) replaces what would otherwise be 50 separate PINN trainings (~50 × 87.3 s ≈ 4,364 s) — roughly an 11× reduction for this dataset, and the saving grows with the number of ICs needed.

PPINN — Strengths & Limitations

Strengths	Limitations
Generalizes to new ICs within the training range, no retraining	Longer training time than a single PINN (more data to cover)
Lowest RMSE/MAE of all core methods in this lab	Larger, harder optimization problem (multi-trajectory batching)
One model replaces many individually-trained PINNs	Cannot extrapolate reliably far outside the training IC range
Still physically consistent (same physics loss as PINN)	Still bounded by the chaotic prediction horizon

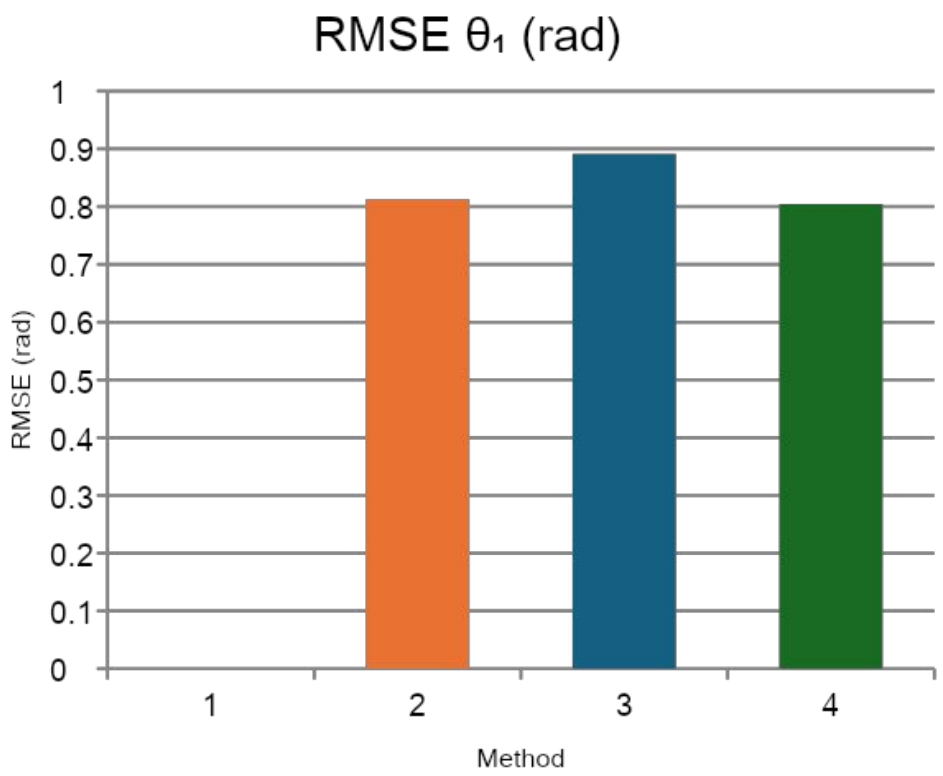
COMPARISON

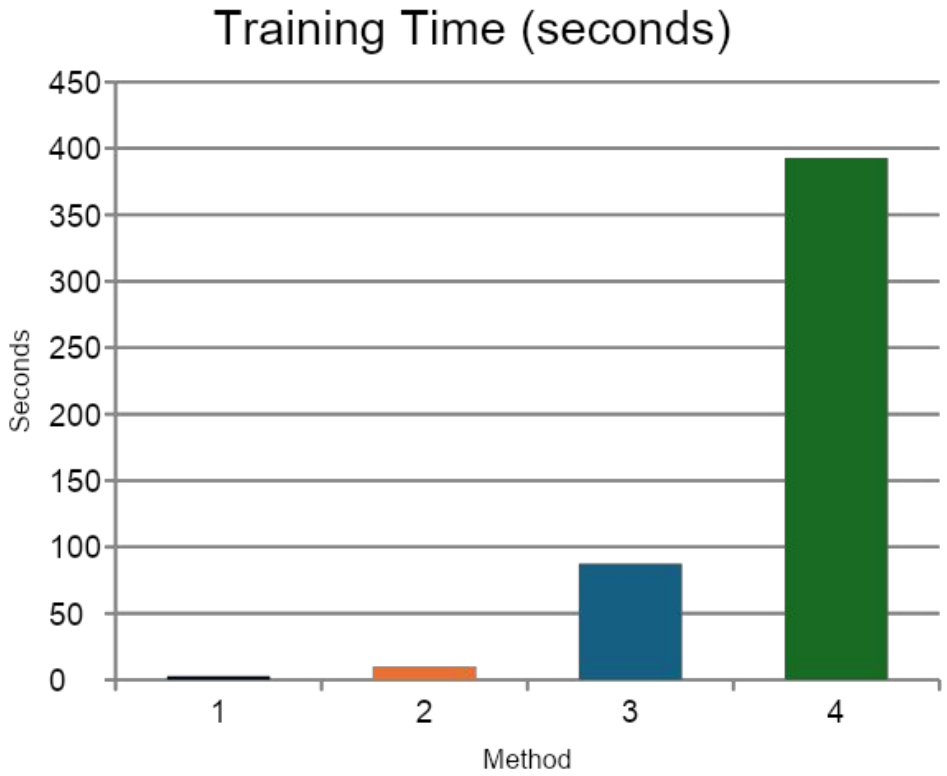
Full Comparison — RK4 vs. MLP vs. PINN vs. PPINN

Method	RMSE θ_1	RMSE θ_2	MAE θ_1	MAE θ_2	Train (s)	Infer 1 (ms)
RK4	0 (ref.)	0 (ref.)	0	0	2.96	—
MLP	0.813	0.968	0.732	0.867	9.8	1.48
PINN	0.891	1.048	0.772	0.913	87.3	1.75
PPINN	0.804	0.931	0.710	0.794	392.7	1.55

COMPARISON

RMSE & Training Time at a Glance





Maynor Ballina
MLAB · ICTP

SECTION 6

Compressing Models — Quantization

What Is Quantization?

Neural-network weights and activations are normally stored as **32-bit floating point** (fp32): 1 sign + 8 exponent + 23 mantissa bits, ≈7 decimal digits of precision.

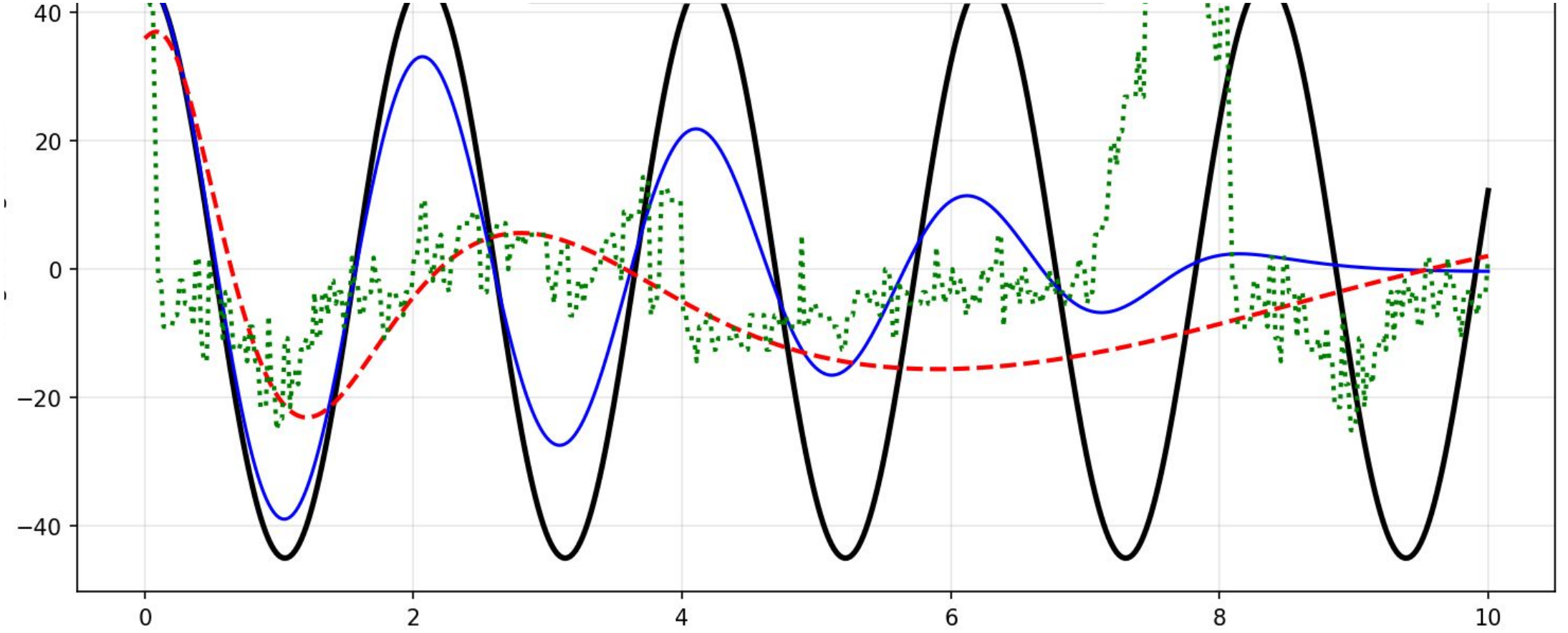
Quantization reduces numeric precision — e.g. to fp16 (1+5+10 bits, ≈3–4 digits) or 8-bit fixed-point/integer — to shrink memory and speed up arithmetic.

Every weight, activation and intermediate product is **rounded** to the nearest representable value at the lower precision.

Format	Mantissa bits	Machine ε
fp32	23	≈1.2×10&supminus;⁷
fp16	10	≈9.8×10&supminus;⁴
int8	— (fixed-point)	scale-dependent, ≈10&supminus;²

Maynor Ballina
MLAB · ICTP

Why Quantize? — Deploying on the HyperFPGA



Quantization Error Meets a Chaotic System

- Rounding every weight/activation to fp16 or int8 is mathematically equivalent to evaluating a **slightly perturbed function** $\tilde{f}(t) = f(t) + \varepsilon(t)$, with $|\varepsilon|$ set by the format's machine precision.
- For this chaotic system, any such perturbation behaves like a tiny change in initial condition — and grows as $\varepsilon \cdot e^{(\lambda t)}$ under the system's positive Lyapunov exponent λ.
- fp16's ε is ~$10^4\times$ larger than fp32's — so the same exponential growth reaches a visibly wrong trajectory **much sooner**, well inside a typical 10 s simulation window.
- Past that point, the quantized network's output **decorrelates** from the fp32 / RK4 reference and looks erratic or "chaotic" — even though every operation is fully deterministic.

Key Insight

Quantization does not *create* chaos. It **reveals and accelerates** the same sensitive-dependence-on-initial-conditions already built into the double pendulum — converting a slow fp32 divergence into a fast, visually noisy fp16/int8 divergence within the same plotted horizon.

Maynor Ballina
MLAB · ICTP

SECTION 7

Other Models — cPINN, gPINN, xPINN

Maynor Ballina
MLAB · ICTP

OTHER MODELS

Conservative PINN (cPINN)

Motivation: a standard PINN can show *energy drift* — its predicted Hamiltonian H(t) slowly grows or decays over long integrations, which is unphysical for a frictionless pendulum.

Adds an **energy-conservation loss** penalizing the variance of the predicted Hamiltonian.

- Only meaningful without friction (b=0); with friction present, energy legitimately dissipates, so λ_energy should be set to 0.

For This Problem	
Advantage	Directly targets long-horizon energy drift, the most physically noticeable PINN failure mode
Disadvantage	Extra hyperparameter (λ_energy) to tune; meaningless once friction/damping is introduced
RMSE θ_1 (measured)	0.837 — between MLP and PINN
Train time (measured)	87.3 s — same order as PINN

ENERGY LOSS

$$L\_energy = Var(H(t)) \approx 0, \quad H = KE_1 + KE_2 + coupling + PE_1 + PE_2$$

Maynor Ballina
MLAB · ICTP

OTHER MODELS

Gradient-Enhanced PINN (gPINN)

Motivation: a standard PINN only enforces the residual R(t)=0 *at* collocation points; it says nothing about the residual's slope in between.

gPINN additionally enforces **dR/dt = 0**, giving the optimizer richer gradient information about the residual itself — typically converging in fewer epochs.

Cost: computing dR/dt requires **3rd-order automatic differentiation**, making each epoch far more expensive to compute.

For This Problem	
Advantage	Needs fewer epochs (8,000 vs. 10,000) to reach comparable loss
Disadvantage	Measured train time 1,557 s — ~18× slower than PINN despite fewer epochs, because each epoch costs far more
RMSE θ_1 (measured)	0.890 — essentially tied with PINN
Best use case	Smoother PDEs where per-epoch cost is lower; less attractive for this stiff, chaotic ODE

EXTRA LOSS TERM

$$L\_total = \ldots + \lambda\_g \cdot \| dR_1/dt \|^2 + \lambda\_g \cdot \| dR_2/dt \|^2$$

Maynor Ballina
MLAB · ICTP

OTHER MODELS

Extended PINN (xPINN)

Motivation: over a long horizon, a chaotic trajectory is hard for one network to represent everywhere at once. xPINN splits time [0,T] into N sub-domains, each with its own smaller sub-network, joined by interface-continuity constraints.

INTERFACE CONDITIONS

$$\theta_i(t_i^+) = \theta_{i+1}(t_i^-) \quad \omega_i(t_i^+) = \omega_{i+1}(t_i^-)$$

For This Problem	
Advantage	Smaller sub-networks (3×32 vs. 4×64) adapt to local complexity; sub-domains parallelize
Disadvantage	Measured train time 2,020 s — highest of all methods; interface loss adds tuning complexity
RMSE θ_1 (measured)	0.891 — comparable to PINN/gPINN

All Seven Methods, Side by Side

Method	RMSE θ_1	MAE θ_1	Train (s)	Infer 1 (ms)
RK4	0 (ref.)	0	2.96	—
MLP	0.813	0.732	9.8	1.48
PINN	0.891	0.772	87.3	1.75
PPINN	0.804	0.710	392.7	1.55
cPINN	0.837	0.744	87.3	1.55
gPINN	0.890	0.770	1,557.2	1.46
xPINN	0.891	0.777	2,019.6	—

WRAP-UP

Conclusions

CONCLUSIONS

What This Lab Demonstrates

- **RK4** remains the accurate, deterministic reference, but must rerun sequentially for every new initial condition.
- **MLP** offers near-instant inference but lacks physical consistency and generalizes poorly beyond its training data.
- **PINN** embeds the governing equations into training, improving physical consistency and cutting data needs, but remains specific to one initial condition.
- **PPINN** learns an entire family of solutions, generalizing across initial conditions without retraining, and achieved the best RMSE/MAE of the core methods in this lab.
- **cPINN, gPINN, xPINN** each target one specific weakness of the standard PINN (energy drift, slow convergence, long-horizon accuracy), at the cost of extra hyperparameters or training time.
- **Quantization** trades accuracy for speed/memory on hardware such as the HyperFPGA, and that accuracy loss is amplified exponentially by the system's chaotic dynamics.
- Across every method, **chaos imposes a fundamental prediction limit**: small errors grow exponentially regardless of solver or architecture. The goal is not to eliminate this error, but to understand how each methodology manages it.

References

- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs. *Journal of Computational Physics*, 378, 686–707.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257.
- Yu, J., Lu, L., Meng, X., & Karniadakis, G. E. (2022). Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. *Computer Methods in Applied Mechanics and Engineering*, 393, 114823.
- Jagtap, A. D., & Karniadakis, G. E. (2020). Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear PDEs. *Communications in Computational Physics*, 28(5), 2002–2041.
- Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.
- Butcher, J. C. (2016). *Numerical Methods for Ordinary Differential Equations* (3rd ed.). Wiley.